

Практична робота

№6

Проектування асинхронної взаємодії: AsyncAPI та подійно-орієнтована архітектура

Дисципліна: Проектування інформаційних систем
Дата виконання: 2026-05-16

1. Назва системи

Система управління онлайн-навчанням (LMS — Learning Management System)

2. Опис предметної області

LMS — це програмна платформа, що забезпечує організацію та проведення онлайн-навчання: реєстрацію студентів на курси, відстеження навчального прогресу, перевірку знань та видачу сертифікатів після успішного завершення курсу.

Ключові учасники бізнес-процесу:

- Студент — проходить курси, виконує завдання, отримує сертифікати.
- Enrollment Service — обробляє реєстрацію студентів на курси.
- Progress Service — відстежує прогрес проходження уроків.
- Certificate Service — генерує цифрові сертифікати після завершення курсу.
- Notification Service — надсилає студентам email-сповіщення.
- Analytics Service — збирає та агрегує навчальну статистику.

Проблема, яку вирішує система: У традиційній синхронній архітектурі кожен сервіс після завершення своєї операції безпосередньо викликає інші сервіси (Enrollment → Progress → Certificate → Notification). Це створює:

- тісне зчеплення (tight coupling) — відмова будь-якого сервісу блокує весь ланцюг;
- блокування запитів — клієнт чекає завершення всього ланцюга операцій;
- труднощі масштабування — додати новий підписник (наприклад, Analytics) без зміни коду видавця неможливо.

3. Обґрунтування вибору подійно-орієнтованої взаємодії

Подійно-орієнтована архітектура (EDA) є доцільним вибором для LMS з таких причин:

Аргумент	Пояснення
Слабке зчеплення	Enrollment Service не знає про існування Notification Service — він лише публікує подію
Асинхронність	Студент отримує відповідь одразу після запису; сповіщення надходить пізніше

Масштабованість	Кожен підписник масштабується незалежно
Розширюваність	Додавання Analytics Service не потребує змін у Enrollment Service
Стійкість	Якщо Notification Service недоступний, повідомлення зберігається в Kafka та буде доставлено після відновлення

4. Перелік подій

Подія 1: StudentEnrolled

Атрибут	Значення
Назва події	StudentEnrolled
Канал (топик)	<code>lms.student.enrolled</code>
Ініціатор (видавець)	Enrollment Service
Підписники	Notification Service, Progress Service, Analytics Service
Бізнес-сенси	Фіксує факт успішного зарахування студента на курс; ініціює привітальний email та початок відстеження прогресу

Момент виникнення	Після успішної транзакції запису студента у базу даних Enrollment Service
-------------------	---

Подія 2: LessonCompleted

Атрибут	Значення
Назва події	LessonCompleted
Канал (топік)	<code>lms.lesson.completed</code>
Ініціатор (видавець)	Progress Service
Підписники	Certificate Service, Analytics Service
Бізнес-сенси	Фіксує завершення окремого уроку студентом; дозволяє Certificate Service визначити, чи завершено весь курс
Момент виникнення	Після запису результатів тесту студента та оновлення запису прогресу

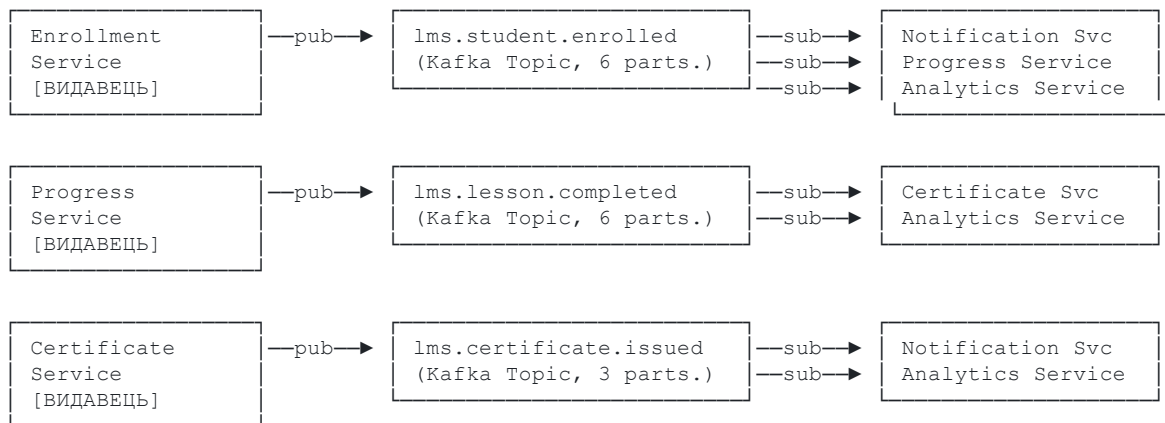
Подія 3: CertificateIssued

Атрибут	Значення
Назва події	CertificateIssued
Канал (топік)	<code>lms.certificate.issued</code>

Ініціатор (видавець)	Certificate Service
Підписники	Notification Service, Analytics Service
Бізнес-сенса	Сигналізує про успішне завершення курсу; Notification Service надсилає студенту email із PDF-сертифікатом
Момент виникнення	Після генерації PDF-сертифіката та запису метаданих у реєстр сертифікатів

5. Схема взаємодії

Схема наведена у файлі `integration_diagram.png`.



Брокер: Apache Kafka (`kafka.lms.example.com:9092`)
Автентифікація: SASL/SCRAM-SHA-256

Центральний елемент — брокер Apache Kafka — розв'язує видавців і підписників у часі та просторі: видавець не знає, хто і коли прочитає повідомлення.

6. Фрагмент специфікації AsyncAPI

Повний файл: `asyncapi.yaml`

Нижче наведено ключові фрагменти для ілюстрації.

Заголовок і сервери

```
asyncapi: '2.6.0'

info:
  title: Learning Management System – Event-Driven API
  version: '1.0.0'

servers:
  production:
    url: kafka.lms.example.com:9092
    protocol: kafka
    description: Продакшн-брокер Apache Kafka
    security:
      - saslScram: []
```

Канал і операції (StudentEnrolled)

```
channels:
  lms/student/enrolled:
    description: Канал для подій запису студента на курс
    subscribe:
      operationId: onStudentEnrolled
      summary: Отримати подію про запис студента на курс
      message:
        $ref: '#/components/messages/StudentEnrolled'
    publish:
      operationId: publishStudentEnrolled
      summary: Опублікувати подію про запис студента на курс
      message:
        $ref: '#/components/messages/StudentEnrolled'
  bindings:
    kafka:
      topic: lms.student.enrolled
      partitions: 6
      replicas: 3
```

Схема повідомлення (StudentEnrolledPayload)

```
components:
  schemas:
    StudentEnrolledPayload:
      type: object
      required:
        - eventId
        - studentId
        - courseId
        - courseTitle
        - enrolledAt
        - enrollmentType
      properties:
        eventId:
          type: string
          format: uuid
        studentId:
          type: string
          pattern: '^STU-\d{5}$'
        courseId:
          type: string
          pattern: '^COURSE-[A-Z]+-\d{3}$'
        enrolledAt:
          type: string
          format: date-time
        enrollmentType:
          type: string
          enum: [free, paid, trial, corporate]
        notifyStudent:
          type: boolean
          default: true
```

7. Пояснення структури повідомлення StudentEnrolled

Для детального аналізу обрано повідомлення StudentEnrolled — перша подія у ланцюзі навчання.

7.1 Поля payload та їх характеристики

Поле	Тип	Обов'язков е	Формат / Обмеження	Призначенн я
eventId	string	Так	UUID v4	Унікальний ідентифікат ор події для ідемпотентн ої обробки
studentId	string	Так	^STU-\d{5}\$	Ідентифікат ор студента у системі
courseId	string	Так	^COURSE-[A-Z]+-\d{3}\$	Ідентифікат ор курсу
courseTitle	string	Так	3–200 символів	Назва курсу (для відображен ня у сповіщенні без додаткового запиту)
enrolledAt	string	Так	ISO 8601 date-time UTC	Точний момент зарахування

<code>enrollmentType</code>	<code>string</code>	Так	enum: free/paid/trial/corporate	Тип зарахування (впливає на логіку сповіщень і статистику)
<code>notifyStudent</code>	<code>boolean</code>	Ні	default: true	Прапорець — чи надсилати email (може бути false для масового імпорту)

7.2 Обов'язкові поля

Поля `eventId`, `studentId`, `courseId`, `courseTitle`, `enrolledAt`, `enrollmentType` є обов'язковими (`required`). Відсутність будь-якого з них означає неповноцінну подію, яку підписники не зможуть обробити коректно.

7.3 Типи даних

- `string (UUID)` — `eventId`: глобально унікальний ідентифікатор, дозволяє підписникам виявляти дублікати.
- `string (pattern)` — `studentId`, `courseId`: регулярний вираз забезпечує консистентний формат ідентифікаторів.
- `string (date-time)` — `enrolledAt`: ISO 8601 із UTC-зоною, уникає проблем із часовими поясами.
- `string (enum)` — `enrollmentType`: чітко обмежений набір значень, несподіване значення є помилкою схеми.
- `boolean` — `notifyStudent`: семантично явний прапорець.

7.4 Помилки через некоректне повідомлення

Помилка	Причина	Наслідок
Відсутнє поле <code>eventId</code>	Генератор події не створив UUID	Підписник не може дедублікувати подію; можливе подвійне нарахування
<code>enrolledAt</code> у неправильному форматі (не ISO 8601)	Часова мітка передана як Unix timestamp	Notification Service не може розпарсити дату; сповіщення не надсилається
<code>enrollmentType</code> поза enum (наприклад, "vip")	Нова версія Enrollment Service без оновлення специфікації	Certificate Service відхиляє повідомлення; студент не отримує запис на курс
<code>studentId</code> не відповідає патерну	Тестові дані потрапили в продакшн	Analytics Service агрегує некоректні записи; статистика спотворена
<code>courseId</code> відсутнє	Помилка серіалізації у видавці	Progress Service не може ініціалізувати запис прогресу; курс "невидимий" для студента

8. Порівняння OpenAPI та AsyncAPI

Критерій	OpenAPI (REST)	AsyncAPI (EDA)
----------	----------------	----------------

Стиль взаємодії	Синхронний — клієнт надсилає запит і чекає відповіді	Асинхронний — видавець публікує подію, підписник реагує незалежно
Основна одиниця опису	<code>endpoint</code> (path + HTTP-метод)	<code>channel</code> (топік або черга)
Роль посередника	Відсутній — клієнт звертається до сервера напряму	Брокер (Kafka, RabbitMQ) — зберігає та доставляє повідомлення
Спосіб обміну даними	Request / Response (синхронна пара)	Event / Message (одностороннє повідомлення: publish / subscribe)
Ініціатор взаємодії	Клієнт (pull-модель)	Видавець або подія у системі (push-модель)
Зв'язаність компонентів	Тісна (tight coupling) — клієнт знає URL сервера	Слабка (loose coupling) — видавець не знає підписників
Гарантія доставки	HTTP-статус коду відповіді	At-most-once / At-least-once / Exactly-once (залежно від налаштувань брокера)
Типові сценарії	CRUD-операції, пошук, аутентифікація	Сповіщення, обробка подій, міжсервісна інтеграція, потокова обробка даних
Інструменти специфікації	Swagger UI, OpenAPI Generator	AsyncAPI Studio, AsyncAPI CLI, Spectral

Протоколи	HTTP/HTTPS	Kafka, AMQP, MQTT, WebSocket, NATS
-----------	------------	------------------------------------

Ключова відмінність: в OpenAPI описується що сервер може зробити у відповідь на запит; в AsyncAPI описується яка подія виникає і хто на неї реагує. REST — це телефонна розмова (обидва мають бути онлайн); EDA — це пошта (відправник і отримувач незалежні).

9. Валідація і контроль якості специфікації

9.1 Spec Validation (структурна валідація)

Інструмент: AsyncAPI CLI (`asyncapi validate`)

Що перевіряє:

- відповідність документа формальній JSON-схемі AsyncAPI 2.6.0;
- наявність усіх обов'язкових полів верхнього рівня (`asyncapi`, `info`, `channels`);
- коректність усіх `$ref` посилань — чи існують компоненти, на які вони вказують;
- типи даних і формати у схемах (`type`, `format`, `enum`, `pattern`);
- синтаксичну коректність YAML.

Встановлення

```
npm install -g @asyncapi/cli
```

Валідація

```
asyncapi validate asyncapi.yaml
```

```
# Очікуваний вивід: "File asyncapi.yaml is valid!"
```

9.2 Linting (перевірка внутрішніх правил і best practices)

Інструмент: Spectral (`spectral lint`)

Чим відрізняється від валідації: Валідація перевіряє синтаксичну коректність — чи документ відповідає специфікації AsyncAPI. Linting перевіряє якість і узгодженість — чи документ відповідає внутрішнім правилам команди та загальноприйнятим best practices.

Приклади Spectral-правил для AsyncAPI:

- `asyncapi-info-contact` — наявність контактної інформації команди;
- `asyncapi-info-license` — наявність ліцензії;
- `asyncapi-message-examples` — повідомлення мають містити приклади (`examples`);
- `asyncapi-operation-description` — операції мають мати текстовий опис;
- `asyncapi-schema-description` — поля схем мають мати `description`.

```
# Встановлення
```

```
npm install -g @stoplight/spectral-cli
```

```
# Lint із офіційним ruleset
```

```
spectral lint asyncapi.yaml --ruleset @asyncapi/spectral-ruleset
```

9.3 Чому обидві перевірки важливі в командній розробці

У командній розробці специфікація є контрактом між командами: порушення специфікації (наприклад, відсутній обов'язковий `$ref`) призводить до неробочої інтеграції. Порушення linting-правил (наприклад, відсутні `examples`) не ламає специфікацію, але ускладнює розуміння контракту іншими розробниками та автоматичну генерацію документації і тестів. Тому валідація та linting є взаємодоповнювальними, а не взаємозамінними.

9.4 CI/CD інтеграція

```
# .github/workflows/asynccapi-check.yml (фрагмент)
```

```
- name: Validate AsyncAPI spec
```

```
  run: asynccapi validate asynccapi.yaml
```

```
- name: Lint AsyncAPI spec
```

```
  run: spectral lint asynccapi.yaml --ruleset @asynccapi/spectral-ruleset
```

Детальний звіт: файл `validation_notes.txt`.

10. Висновки

У ході виконання практичної роботи було:

1. Обрано прикладну систему — LMS (Learning Management System) — з чітким бізнес-контекстом та реальними взаємодіями між сервісами.
2. Визначено три події (`StudentEnrolled`, `LessonCompleted`, `CertificateIssued`), для кожної встановлено видавця, підписників та бізнес-сенси. Це показало, як EDA дозволяє декомпонувати складний бізнес-процес на незалежні події.
3. Побудовано схему взаємодії у форматі «видавець → брокер (Kafka) → підписник», яка наочно демонструє слабе зчеплення між компонентами системи.
4. Складено специфікацію AsyncAPI 2.6.0 (`asynccapi.yaml`) з описом серверів, каналів, операцій `publish/subscribe`, повідомлень і схем `payload`. Специфікація містить приклади значень, обмеження типів і патерни для ідентифікаторів.
5. Виконано порівняння OpenAPI та AsyncAPI, яке показало принципову відмінність між синхронною (`request/response`) та асинхронною (`event/message`) моделями взаємодії.

6. Описано підхід до валідації: структурна перевірка через `asyncapi validate` та якісна перевірка через `spectral lint` є двома незалежними, але взаємодоповнювальними практиками контролю якості специфікації.

Практична робота демонструє, що AsyncAPI є інженерним артефактом, а не просто документацією: на основі специфікації можна автоматично генерувати код (scaffold), документацію та тести, що прямо відповідає підходу Design-First у MuleSoft Design Center та Code Builder.